

```

library(forecast)
library(fpp2)
library(tidyverse)
library(ggplot2)

#Example 1

data <- forecast::wineind

autoplot(data)
#what do you see?
#Trend - non-linear, increasing: might have to zoom in on a
certain window to investigate linearity
#seasonality: monthly observations, length of seasonal
variation?
#cyclicalilty: potential cyclicalilty from around 1987

autoplot(data)+
  autolayer(ma(data,12))
#moving average smoothing to remove the seasonal component

data %>% ggAcf()#significant autocorrelations at lags 6, 12,
18, 24 - can you deduce the seasonal component from the ACF?
data %>% ggPacf()#excess autocorrelation only really notable
at lag 6 and 12 now (18 and 24 much diminished)

#investigate only the linear part of the time series by
limiting it to a smaller window:
data <- window(data, end = c(1986,12))
autoplot(data)+
  autolayer(ma(data,12))

#transformations required to address issues with stationarity?
#stabilise the variation:check the size of lambda
lambda <- BoxCox.lambda(data)
lambda

#how many times do we need to difference?
nsdiffs(data)

#apply the transformations and assess for stationarity:
newdat <- data %>% BoxCox(lambda) %>% diff(12)
autoplot(newdat)

```

```
#looks stationary, but maybe not a white noise process (mean not exactly 0)
```

```
#look at the classical decomposition
#adjusted SI with decompmod$figure
#unless otherwise specified, assume the first value is for S1, the second value is for S2 etc.
#read off estimated values for detrended, deseasonalised data and random variation
decompmod<-decompose(data, type = "multiplicative")
decompmod
plot(decompmod)
```

```
#split data into a test and train set
train <- window(data, end=c(1984,12))
test <- window(data, start = c(1985,1))
```

```
mod1 <- hw(train, h=24) #fit and forecast with holt-winters (trend + seasonality)-expected best strategy overall (not considering arima here)
mod2 <- rwf(train, drift = TRUE, h=24)#fit and forecast with drift (trend)- best long-term simple forecasting strategy
mod3 <- snaive(train, h = 24)#fit and forecast with seasonal naive (seasonality)- best short term simple forecasting strategy
```

```
#can you discuss all four elements of the residuals? Give ALL the details
checkresiduals(mod1)
checkresiduals(mod2)
checkresiduals(mod3)
```

```
#output, smoothing parameters, initial states etc of holt-winters
summary(mod1)
```

```
#apply to test data and evaluate performance with regards to forecasting accuracy - lowest RMSE and MAE indicates better model
accuracy(mod1, test)
accuracy(mod2, test)
accuracy(mod3, test)
```

```
#visually assess the forecasting accuracy by comparing it to
the actual observations
```

```
autoplot(data)+
  autolayer(mod1, PI=FALSE)+
  autolayer(mod2, PI=FALSE, color = "green")+
  autolayer(mod3, PI = FALSE, color = "red")
```

```
#Example 2
```

```
#Another data set for ARIMA modelling:
```

```
data2 <- fpp2::marathon
```

```
autoplot(marathon)#Describe the components of the time series?
```

```
#stabilise variance:
```

```
lam <- BoxCox.lambda(data2)
data2_boxcox <- BoxCox(data2, lam)
autoplot(data2_boxcox)
```

```
#first-order differencing, no seasonality, at lag g=1
```

```
data2_boxcox_diff <- diff(data2_boxcox)
autoplot(data2_boxcox_diff)
```

```
#look at acf and pacf to decide how to select MA(q) and AR(p)
```

```
data2_boxcox_diff %>% ggAcf()
data2_boxcox_diff %>% ggPacf()
```

```
#one of the general cases of ARIMA
```

```
#in both acf and pacf no clear indication of which models to fit - we should
consider fitting a few lower-order models
```

```
#but R has a neat function that selects the best model on its
own:
```

```
arima1<-auto.arima(data2_boxcox_diff)
```

```
arima1
```

```
arima2<-arima(data2_boxcox_diff, order = c(2,0,2)) #already
differenced, just know d = 1
```

```
arima2
```

```
## AIC = -1637.36 vs AIC = -1638.95 (better model is arima1)
```

```
#suggest model vs lower order models and compare the AIC's
```

```
checkresiduals(arima1)
```

```
checkresiduals(arima2)
```

→ ARIMA(2,1,2)

↳ could also fit ARIMA(1,1,1),
ARIMA(2,1,1),
ARIMA(1,1,2)
etc.

